U

ASSESSMENT COVERSHEET

Attach this coversheet as the cover of your submission. All sections must be completed.

Section A: Submission Details

Programme : BACHELOR OF INFORMATION TECHNOLOGY IN COMPUTER SYSTEM
Course Code & Name : IKB21503 - SECURE SOFTWARE DEVELOPMENT
Course Lecturer(s) : MADAM MARDIANA BINTI MAHARI
Submission Title : PROJECT
Deadline : Day 18 Month 6 Year 2026 Time 11:59
Penalties :
- 5% will be deducted per day to a maximum of four (4) working days, after which the submission will **not** be accepted.
- Plagiarised work is an Academic Offence in University Rules & Regulations and will be penalised accordingly.

Section B: Academic Integrity

Tick (√) each box below if you agree:

√ I have read and understood the UniKL's policy on Plagiarism in University Rules & Regulations.
√ This submission is my own, unless indicated with proper referencing.
√ This submission has not been previously submitted or published.
√ This submission follows the requirements stated in the course.

Section C: Submission Receipt

(must be filled in manually)

Office Receipt of Submission

Date & Time of Submission	Student Name	Lab Class	Task/Role - Contribution
18/06/2026 23:59 PM	MUHAMMAD HAFIZ DANIAL BIN MOHD ZAKI 52215125106	L02-B02	Application Development 100%
	AHMAD DANIAL HAZIQ BIN IBRAHIM 52215125773	L02-B02	Security Testing & Mitigation Development 100%

Student Receipt of Submission

This is your submission receipt, the only accepted evidence that you have submitted your work. After this is stamped by the appointed staff & filled in, cut along the dotted lines above & retain this for your record.

Date & Time of Submission	Student Name	Lab Class	Task/Role - Contribution
18/06/2026 23:59 PM	MUHAMMAD HAFIZ DANIAL BIN MOHD ZAKI 52215125106	L02-B02	Application Development 100%
	AHMAD DANIAL HAZIQ BIN IBRAHIM 52215125773	L02-B02	Security Testing & Mitigation Development 100%

Contents

1. Executive Summary

This project was developed with highly secure Task Management Web Application with OWASP-Compliant Development Practices using Python Django framework. The application was engineered to carefully follow OWASP Top 10 and OWASP ASVS guidelines. The application has strong input validation, strict access controls, robust error handling, encrypted passwords and log everything if anything happened. We used a shared GitHub pipeline for our development and did thorough security checks during staging phases to identify and address logic and validation gaps. We successfully identify and mitigate input bypass validation and Stored Cross-Site Scripting (XSS) before that cause problems. The result is optimized production -hardened software release that align with NIST SSDF standards.

2. Introduction

2.1 Background

The application is designed as a collaborative Task Management System. The structure includes user registration, session-based authentication, different access levels for regular users and admins, fully validated task operations, customizable user profiles and administrative audit logs for monitoring.

2.2 Project Scope

The scope is to design, develop and secure a web-based collaborative Task Management System utilizing Python Django framework. The development cycle includes secure coding practice across frontend and backend layers, along with automated and manual security testing to identify and mitigate the system risk. The functional and technical boundaries include:

- **User Management and Role-Based Access Control (RBAC):** Implementing a secure user registration and authentication pathways with custom role to separation between standard "Normal Users" and "Admins."
- **Secure Task CRUD Operations:** Developing a fully validated Task Management module that allow user to create, view, edit and delete their own task while preventing unauthorized cross user modification through IDOR protection.
- **User Profile Management:** Allowing authenticated user to manage their profiles including secure file uploads for profiles picture using Django's managed file infrastructure.
- **Security Auditing and Logging:** Record all security-sensitive events such as logins, registration attempts, task management profiles update in read-only administrative Audit Log module.

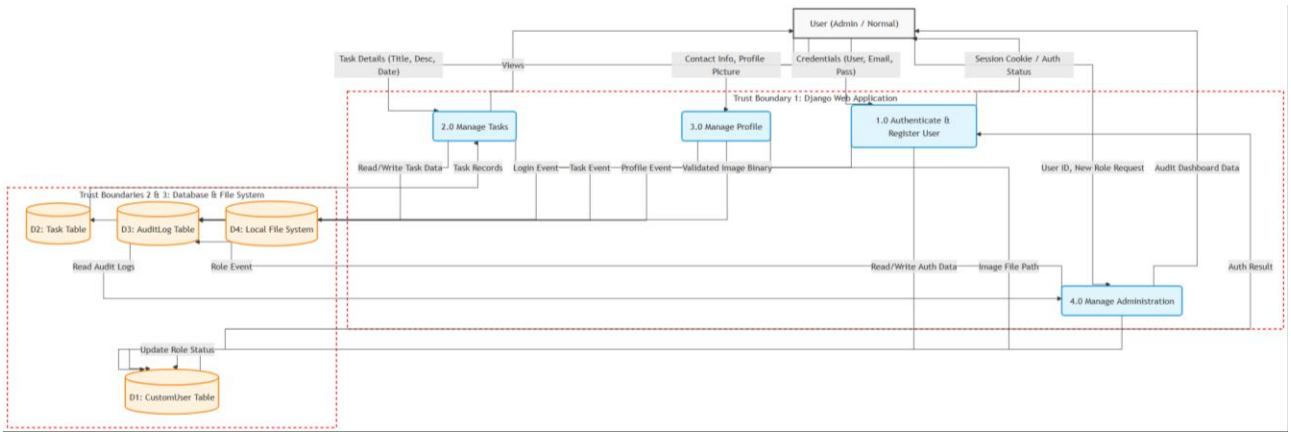
- **Secure Software Development Lifecycle (SSDLC) Pipelines:** Manage version control on a collaborative GitHub repository and implementing secret separation and running vulnerability assessment tools to remediate risk prior to final production release.

2.3 Objective

The primary objective of this project is:

- **Develop secure web application:** Implement a robust and fully functional Task Management system with secure architecture configurations with Django framework.
- **Apply OWASP Top 10 & ASVS Standards:** Implement a technical control that align with OWASP Top 10 security risks and OWASP Application Security Verification Standard (ASVS) criteria in authentication, input validation, access control and session security.
- **Ensure Injection-Free Coding:** Prevent all type of injection attack including SQL Injection, HTML Injection and Stored Cross-Site Scripting by applying parameterized Object-Relational Mapping (ORM), built-in template auto-escaping, and strict server-side validation.
- **Maintain Secure Coding and Configuration Hygiene in GitHub:** Practice secure repository management by decoupling secret from version control using external '.env' variable and enforcing peer on version-controlled code.
- **Practice DevSecOps Basics:** Applying security automation by conducting Software Component Analysis (SCA) dependency testing, Static Application Security Testing (SAST) source code scanning and manual penetration testing to demonstrate a how vulnerabilities were mitigated.

2.4 Data Flow Diagram



3. Secure Coding Implementation

3.1. Input Validation

```python
class RegisterForm(forms.Form):
    username = forms.CharField(
        max_length=150,
        validators=[
            RegexValidator(
                regex='^[a-zA-Z0-9_]+$',
                message='Username can only contain letters, numbers, and underscores'
            )
        ]
    )
    email = forms.EmailField()
    password = forms.CharField(
        widget=forms.PasswordInput,
        min_length=8,
        validators=[MinLengthValidator(8, 'Password must be at least 8 characters')]
    )
```

Figure 1 forms.py

The application uses strict input validation using Django forms. Use RegexValidator to strict only alphanumeric characters for usernames to prevent injection attack and use robust MinLengthValidator to maintain baseline credential complexity.

3.2. Authentication & Session Security

```python
SESSION_COOKIE_AGE = 1800
SESSION_COOKIE_HTTPONLY = True
SESSION_EXPIRE_AT_BROWSER_CLOSE = True
CSRF_COOKIE_HTTPONLY = True
```

Figure 2 settings.py

To mitigate session hijacking risk, the application implement session expired after 30 minutes using ESSION_COOKIE_AGE setting and add HTTPONLY flags to prevent client-side scripts from accessing session and CSRF tokens.

3.3. Access Control

```python
@login_required
def task_edit(request, pk):
    task = get_object_or_404(Task, pk=pk)
    if request.user.role != 'admin' and task.created_by != request.user:
        messages.error(request, 'You do not have permission to edit this task')
        return redirect('dashboard')
```

Figure 3 views.py

To mitigate broken access control and unauthorized data exposure, the system implement strict data isolation task data strategy by dynamically filtering database queries using the session authenticated user ID. This is to ensure that standard user are strictly limited to view or edit their own task objects.

3.4. Error Handling

```python
def login_view(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        if not username or not password:
            messages.error(request, 'Please enter both username and password')
            return render(request, 'tasks/login.html')

        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)
            log_audit(user, 'LOGIN_SUCCESS', request)
            messages.success(request, f'Welcome back, {user.username}!')
            return redirect('dashboard')
        else:
            log_audit(None, 'LOGIN_FAILED', request, f'Failed login attempt for {username}')
            messages.error(request, 'Invalid username or password')
```

Figure 4 views.py

To prevent user enumeration vulnerabilities, the system handles invalid credentials request consistently by returning a generic string ("Invalid username or password") instead of identifying which credentials component is incorrect.

3.5. Sensitive Data Protection

```
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
        'OPTIONS': {
            'min_length': 8,
        }
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

Figure 5 settings.py

Password Is kept strong by using Django's core authentication validator that check them against common words, frequent phrases and numeric pattern before hashing them.

3.6. File Upload Security

```
@login_required
def profile_edit(request):
    if request.method == 'POST':
        form = ProfileForm(request.POST, request.FILES, instance=request.user)
        if form.is_valid():
            form.save()
            log_audit(request.user, 'PROFILE_UPDATED', request)
            messages.success(request, 'Profile updated successfully!')
            return redirect('profile')
    else:
        form = ProfileForm(instance=request.user)
    return render(request, 'tasks/profile_edit.html', {'form': form})
```

Figure 6 views.py

```
class ProfileForm(forms.ModelForm):
    class Meta:
        model = CustomUser
        fields = ['first_name', 'last_name', 'email', 'phone', 'profile_picture']
```

Figure 7 forms.py

The application implements controlled profile images uploads with Django's ImageField architecture. File processing is handled through Django's managed upload framework

instead of direct accessing the system, which prevent arbitrary file execution attacks. The upload only available to authenticated user for managing their profiles to reducing the risk of public upload vulnerabilities.

3.7. Configuration Security

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Figure 8 settings.py

The application implements basic security protection into the app using Django's built-in security headers. The CsrfViewMiddleware stop Cross-Site Request Forgery attacks and the XFrameOptionsMiddleware blocks Clickjacking by preventing frame rendering to mitigate Clickjacking techniques.

3.8. Logging & Monitoring

```
class AuditLog(models.Model):
    """Audit log for security events"""
    ACTION_CHOICES = (
        ('LOGIN_SUCCESS', 'Login Success'),
        ('LOGIN_FAILED', 'Login Failed'),
        ('LOGOUT', 'Logout'),
        ('TASK_CREATED', 'Task Created'),
        ('TASK_UPDATED', 'Task Updated'),
        ('TASK_DELETED', 'Task Deleted'),
        ('ROLE_CHANGED', 'Role Changed'),
        ('PROFILE_UPDATED', 'Profile Updated'),
    )
```

Figure 9 models.py

```python
def log_audit(user, action, request, details=''):
    AuditLog.objects.create(
        user=user,
        action=action,
        ip_address=request.META.get('REMOTE_ADDR'),
        details=details
    )
```

Figure 10 views.py

The application implements centralized security event logging system with AuditLog model. It records important activities like login attempt, task changes and profiles modification along with user IP Address. This data will help tack what is happening, monitor security and investigate any incidents more effectively.

3.9. Output Encoding

```
<tr>
    <td style="font-weight: 600;">{{ task.title }}</td>
    <td style="color: #6b7280;">{{ task.description|truncatechars:50 }}</td>
    <td>
```

Figure 11 dashboard.html

The application implements Django's template engine for secure HTML output encoding when displaying data such as task title and description. By avoiding unsafe modifier '|safe' filter, user-supplied content is parsed strictly as safe as plain-text strings instead of executable.

4. Security Testing & Results

No.	Vulnerability Identified	STRIDE Category	Severity
1	Stored Cross-Site Scripting (XSS) & HTML Injection	Tampering	High
2	Broken Authentication & Improper Error Handling	Spoofing	Medium

4.1. Manual Code Review Checklist

No.	Item to Check	Description	SSDF Practice	Status
1	Input Validation	Check that all user input is properly validated using whitelists or parameterized inputs to prevent injection or	P.W.5	Passed

		data corruption.		
2	Authentication & Session Management	Ensure secure login process, strong password policy, and session timeouts are implemented to prevent unauthorized access.	P.W.5	Passed
3	Access Control	Verify that sensitive resources are only accessible to authorized users according to their roles.	P.W.5	Passed
4	Error Handling	Confirm that the application handles errors with try/catch blocks and does not expose stack traces or system information.	P.W.6	Passed
5	Sensitive Data Protection	Check that credentials, API keys, and tokens are not hard-coded or logged in plaintext.	P.W.4	Passed
6	File Upload Security	Validate file type and size before upload; ensure uploaded files are stored outside the web root to prevent RCE attacks.	P.W.7	Passed
7	Configuration Security	Make sure environment variables, API keys, and dependency versions are securely configured and updated.	P.W.6	Passed
8	Logging & Monitoring	Verify that security events such as failed logins or access violations are logged and monitored properly.	P.W.6	Passed
9	Dependency Management	Ensure all third-party libraries are up-to-date and verified from trusted sources.	P.W.6	Passed

10	Output Encoding / Escaping	Confirm that all outputs are encoded before rendering to prevent XSS attacks.	P.W.5	Passed

4.2. Software Component Analysis

```
C:\Users\Denni\Documents\UNIKL\SEM2\SSD\Group\secure-task-app>venv\Scripts\activate

(venv) C:\Users\Denni\Documents\UNIKL\SEM2\SSD\Group\secure-task-app>snyk test --file
=requirements.txt

Testing C:\Users\Denni\Documents\UNIKL\SEM2\SSD\Group\secure-task-app...

Organization:      denniiyyy
Package manager:   pip
Target file:       requirements.txt
Project name:      secure-task-app
Open source:       no
Project path:      C:\Users\Denni\Documents\UNIKL\SEM2\SSD\Group\secure-task-app
Licenses:          enabled

✔Tested 5 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

Figure 12 snyk CLI

Automated dependency scanning using Snyk to analysed third-party Python packages specified in requirements.txt. This process checks installed libraries against Snyk's vulnerability database to identify known security issues, outdated packages or license compliance risks. This process helps us proactively patch upstream components, monitor package dependencies and protect against supply chain exploits targeting the application server.

4.3. Static Analysis

Tools Used : Snyk

Target File : taskproject/settings.py (Line 6)

Vulnerability Checked : Hardcoded Non-Cryptographic Secret (CWE-547)

During the staging scan, snyk flagged a high-severity vulnerability pointing out a hardcoded SECRET_KEY within settings.py. To resolving this issue using django-environ library to fetch these configuration from an external untracked .env file to blocks unauthorized users on GitHub from viewing our cryptographic keys.

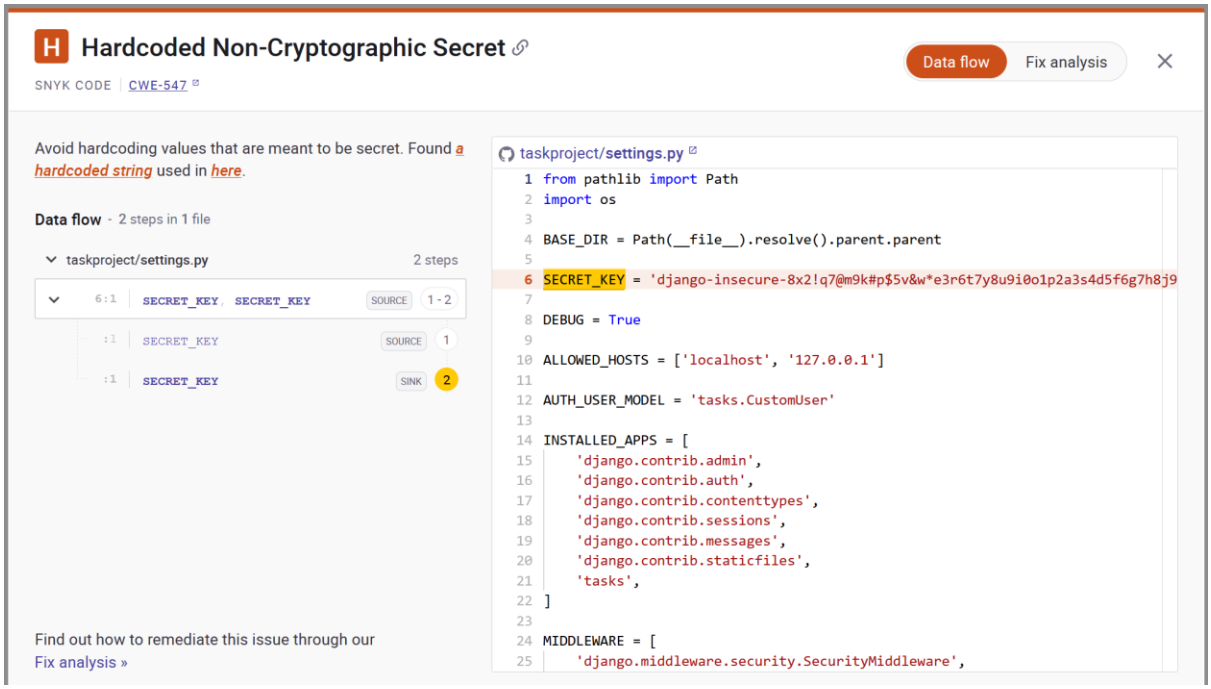


Figure 13 Snyk Scan

Before Mitigation

```python
from pathlib import Path
import os

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'django-insecure-8x2!q7@m9k#p$5v&w*e3r6t7y8u9i0o1p2a3s4d5f6g7h8j9k0l'

DEBUG = True

ALLOWED_HOSTS = ['localhost', '127.0.0.1']

AUTH_USER_MODEL = 'tasks.CustomUser'
```

Figure 14 settings.py

Sensitive variables were hardcoded directly inside our Django settings module as flagged in our Snyk scan.

After Mitigation

```
SECRET_KEY=django-insecure-8x2!q7@m9k#p$5v&w*e3r6t7y8u9i0o1p2a3s4d5f6g7h8j9k0l
DEBUG=True
```

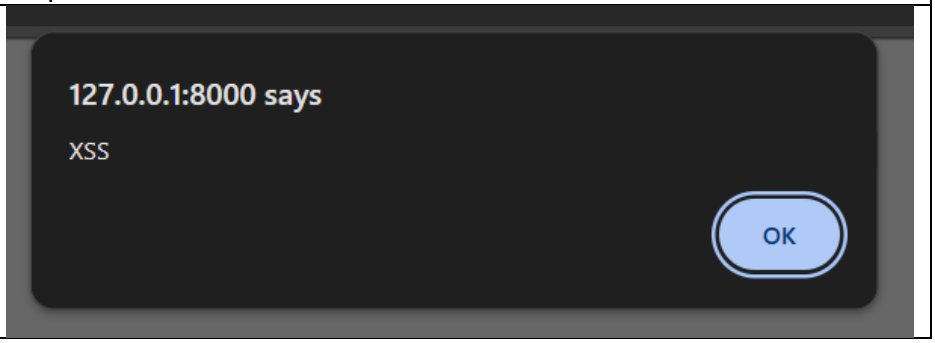
Figure 15 .env

```python
from pathlib import Path
import os
import environ

BASE_DIR = Path(__file__).resolve().parent.parent

env = environ.Env(
    DEBUG=(bool, False)
)

environ.Env.read_env(os.path.join(BASE_DIR, '.env'))

SECRET_KEY = env('SECRET_KEY')

DEBUG = env('DEBUG')
```

Figure 16 settings.py

To resolve this vulnerable, we decide to use django-environ for our secret management. The settings key and debug configurations are now fetched dynamically from an isolated .env environment file.

4.4. Dynamic Testing (OWASP ZAP / Burp Suite)

Vulnerability identified:	Stored Cross-Site Scripting (XSS) & HTML Injection	Threat Category	Tampering
Vulnerable URL/Parameters	http://127.0.0.1:8000/tasks/dashboard/		
Severity:	High		
Steps performed to identify the vulnerability:	Step 1: Intercept the task creation POST request using Burp Suite Proxy and sent it to Repeater tab. Step 2: Modify the task title parameter payload to IsThisSecure (for HTML Injection) or <script>alert('XSS')</script> (for Stored XSS) and forward the request. Step 3: Refresh the dashboard interface		
Screenshots			

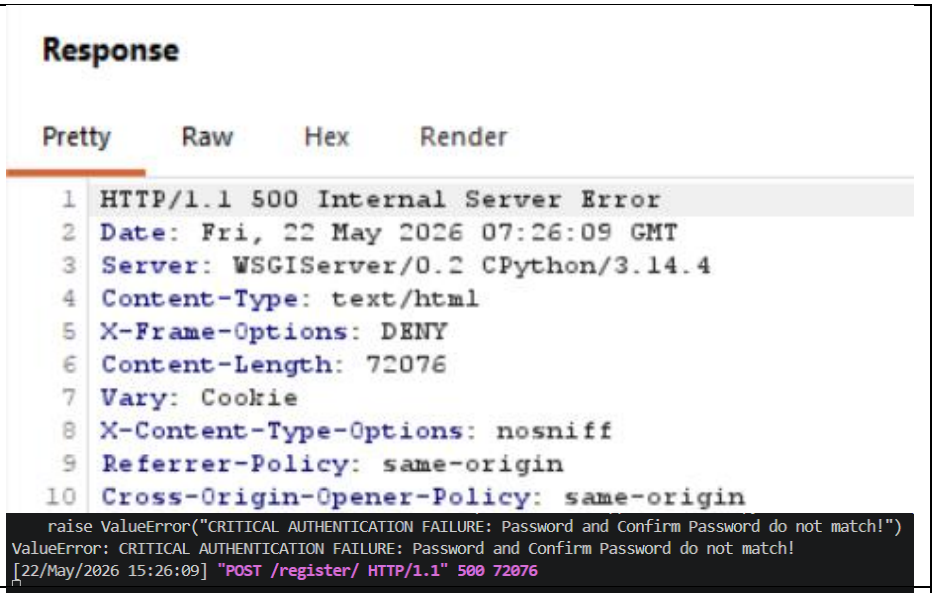
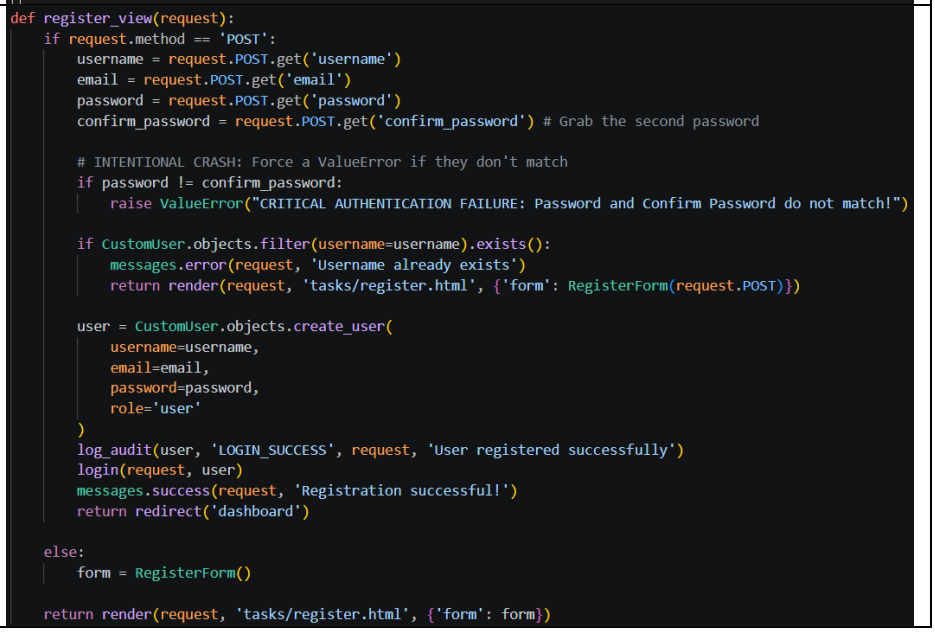
	testing2 testing In_Progress IsThisSecure testing In_Progress

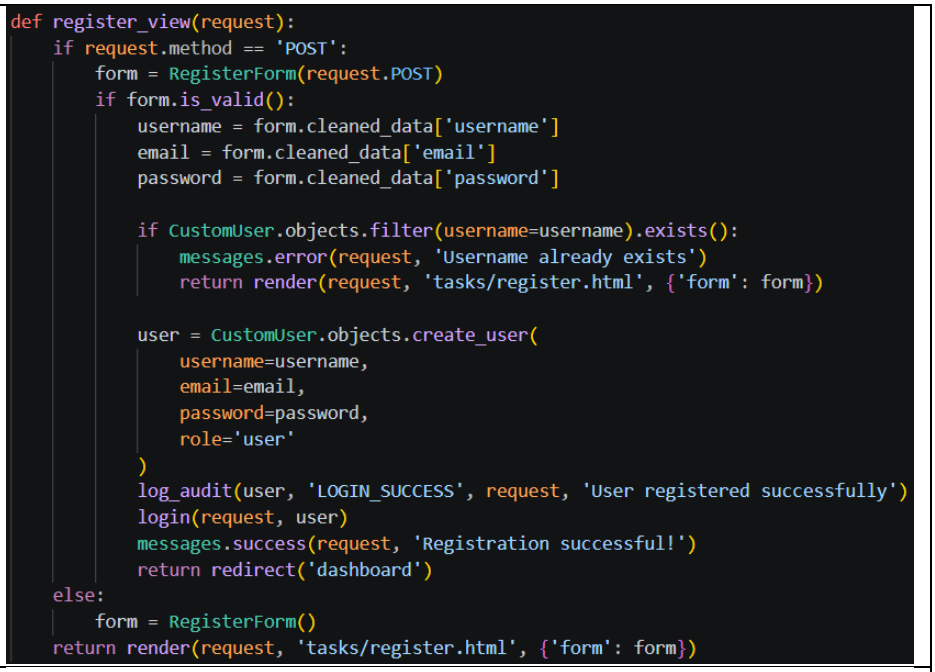
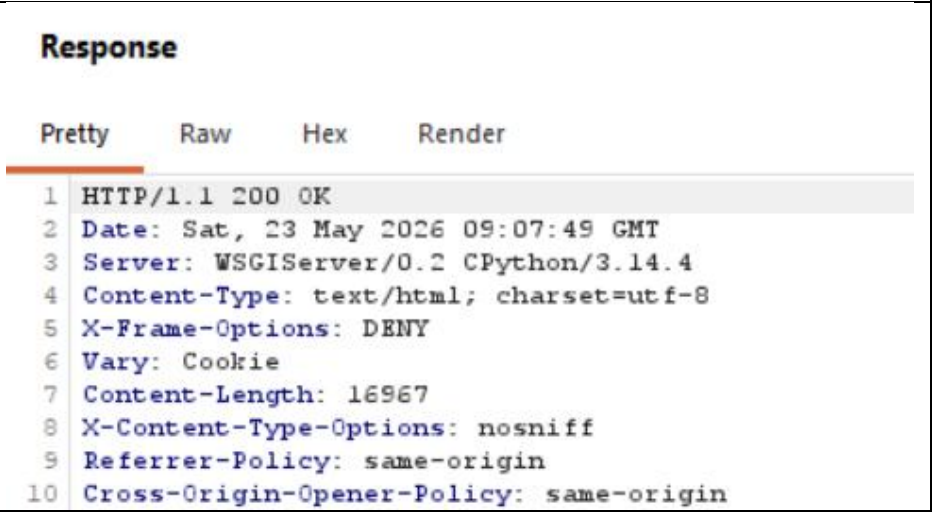
Faulty/ Unsecure coding (dashboard.html)	`{% for task in tasks %}`
	`<tr>`
	`<td style="font-weight: 600;">`
	`{% autoescape off %}`
	`{{ task.title }}`
	`{% endautoescape %}`
	`</td>`
	`<td style="color: #6b7280;">{{ task.description|truncatechars:50 }}</td>`
	`<td>`
	`<span class="task-status status-{{ task.status }}">`

Remediation/Mitigation	`{% for task in tasks %}`
	`<tr>`
	`<td style="font-weight: 600;">{{ task.title }}</td>`
	`<td style="color: #6b7280;">{{ task.description|truncatechars:50 }}</td>`
	`<td>`
	`<span class="task-status status-{{ task.status }}">`

Mitigation Proof	title=<script>alert('XSS')</script> Tesing in working In_Progress testing2 testing In_Progress IsThisSecure testing In_Progress

Vulnerability identified:	Broken Authentication & Improper Error Handling	Threat Category	Spoofing
Vulnerable URL/Parameters	http://127.0.0.1:8000/register/		
Severity:	Medium		
Steps performed to identify the vulnerability:	Step 1: Open registration page and enter registration details. Step 2: Intercept the outgoing registration POST request using Burp Suite Proxy and send it to the Repeater tab. Step 3: Under the Request panel in Repeater, manually change the parameter values so that confirm_password mismatches the primary password field.		

Screenshots	
Faulty/ Unsecure coding	

Remediation/ Mitigation	
Mitigation Proof	

```python
def register_view(request):
    if request.method == 'POST':
        form = RegisterForm(request.POST)
        if form.is_valid():
            username = form.cleaned_data['username']
            email = form.cleaned_data['email']
            password = form.cleaned_data['password']

            if CustomUser.objects.filter(username=username).exists():
                messages.error(request, 'Username already exists')
                return render(request, 'tasks/register.html', {'form': form})

            user = CustomUser.objects.create_user(
                username=username,
                email=email,
                password=password,
                role='user'
            )
            log_audit(user, 'LOGIN_SUCCESS', request, 'User registered successfully')
            login(request, user)
            messages.success(request, 'Registration successful!')
            return redirect('dashboard')
    else:
        form = RegisterForm()
    return render(request, 'tasks/register.html', {'form': form})
```

Response

Pretty Raw Hex Render

```
1  HTTP/1.1 200 OK
2  Date: Sat, 23 May 2026 09:07:49 GMT
3  Server: WSGIServer/0.2 CPython/3.14.4
4  Content-Type: text/html; charset=utf-8
5  X-Frame-Options: DENY
6  Vary: Cookie
7  Content-Length: 16967
8  X-Content-Type-Options: nosniff
9  Referrer-Policy: same-origin
10 Cross-Origin-Opener-Policy: same-origin
```

4.5. Penetration Test Summary

The dynamic penetration testing phase focused on evaluating the application runtime behaviour against common attack vectors from the OWASP Top 10 framework. The testing was conducted on isolated local staging environment using combination of automated vulnerability scanning via Burp Suite. The main objective was to validate whether the implemented security controls effectively restricted unauthorized data manipulation or system compromise under realistic attack conditions.

The assessment successfully exposed two critical runtime anomalies during the initial testing phase:

- **Stored Cross-Site Scripting (XSS) & HTML Injection:** By intercepting the task creation POST request, payload elements including ‘<h1>IsThisSecure</h1>’ or ‘<script>alert('XSS')</script>’ were injected into application parameter. During initial

testing, the application template system executed the tags, generating an active browser alert box component and altering layout design parameters. This confirmed that output escaping controls had been bypassed in the staging iteration of dashboard.html.

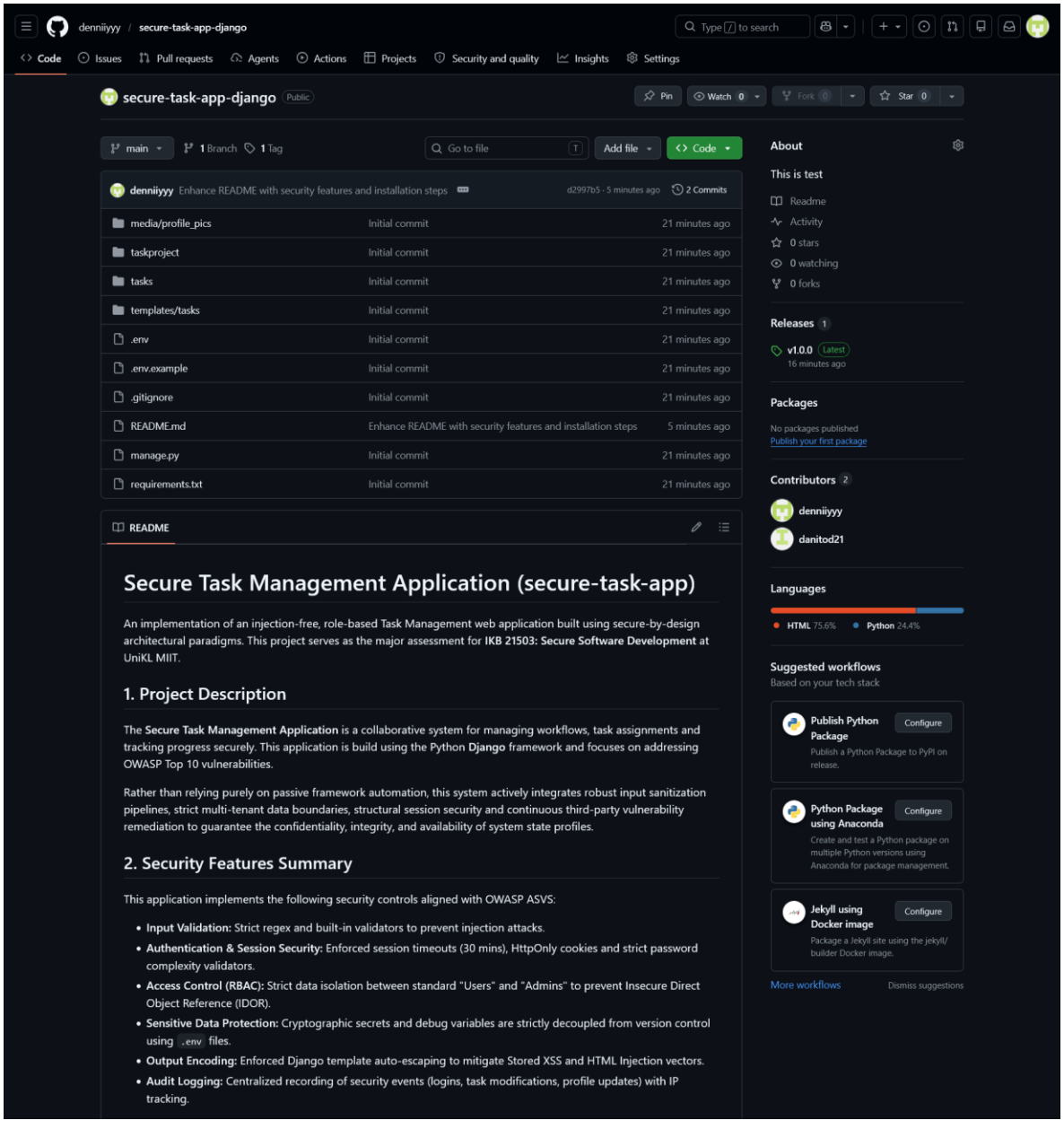
- **Improper Error Handling & Logic Flaws:** Manual parameter tampering was performed on the registration interface by sending mismatched passwords values across the proxy stream. The initial application version failed to catch this data structural mismatch through validation code paths and causing the unhandled backend with that returned a raw HTTP/1.1 500 Internal Server Error page error instead of handling the validation properly.

Remediation and Final Verification Status: Following the discovery of these flaws, a systematic code remediation phase was initiated to bring the application into full compliance with OWASP ASVS guidelines. The explicit bypass blocks ({% autoescape off %}) were removed from the template engine to restore Django's secure-by-default contextual HTML encoding infrastructure. Additionally, the user registration backend views were fixed to handle form exceptions properly and return readable error messages instead of crashing.

A comprehensive testing was completed to verify the effectiveness of these fixes. The application successfully achieved a secure status across all tested modules:

- Injected script strings and HTML elements are parsed as plaintext characters mitigating any risk of arbitrary browser script execution.
- Submitting malicious payloads via network manipulation proxies results in controlled server rejections instead of crashing the server.

5. GitHub Repository & Version Control



6. GitHub Repository Details

- Repository name: secure-task-app-django
- Link to repository: https://github.com/denniiyyy/secure-task-app-django
- Team members' GitHub usernames:
    1. danitod21 (MUHAMMAD HAFIZ DANIAL BIN MOHD ZAKI)
    2. denniiyyy (AHMAD DANIAL HAZIQ BIN IBRAHIM)

- Video Link: https://drive.google.com/drive/folders/1p8dwj-kXLF4a-3MoFYuSrP4TbQWjJbm_?usp=drive_link

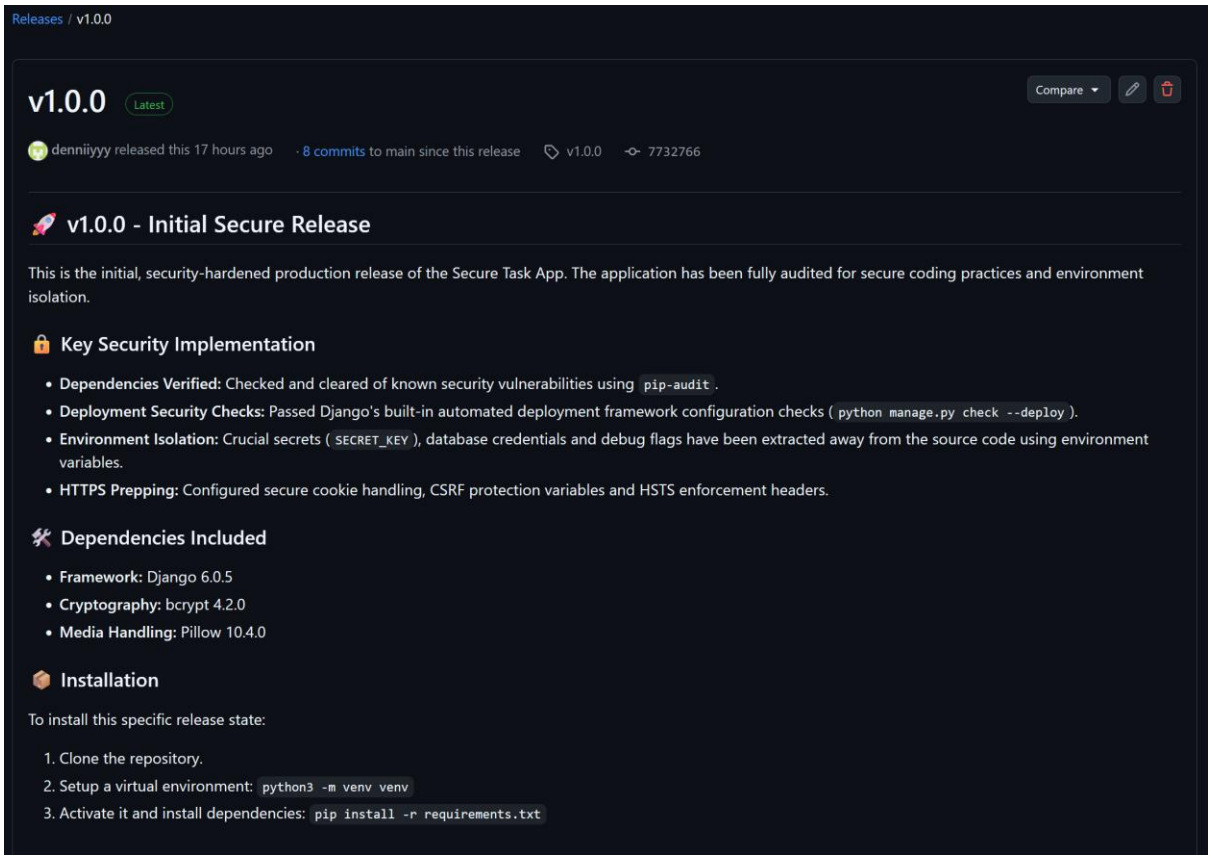
7. Repository Structure

```
secure-task-app/
├── taskproject/            # Django project configuration directory
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py         # Core project settings
│   ├── urls.py             # Root routing configurations
│   └── wsgi.py
├── tasks/                  # Main application app (Task management logic)
│   ├── migrations/         # Database migration files
│   ├── __init__.py
│   ├── admin.py            # Admin interface definitions
│   ├── apps.py
│   ├── models.py           # Database schemas/models (e.g., Task, User)
│   ├── tests.py            # Automated tests for application logic
│   └── views.py            # Request/response logic (Controllers)
├── templates/              # Shared HTML templates/frontend components
├── venv/                   # Local Python virtual environment (Excluded from Git)
├── db.sqlite3              # Local development database (Excluded from Git)
├── manage.py               # Django management command-line utility
├── README.md               # Project documentation and setup guide
└── requirements.txt        # Third-party Python dependencies (SCA Target)
```

Figure 17 Repository Structure

8. README Documentation

8.1. GitHub Release



9. Results & Discussion

During the staging phase of DevSecOps development phase, our team combined automated Static Application Security Testing (SAST) with active DAST using Burp Suite. This integration of dynamic intercept penetration test that allow us to discover logic flaw and output escaping misconfigurations that standard code syntax validators missed. The application testing uncovered two critical security issue:

- **Output Encoding Deficiencies (Stored XSS & HTML Injection):** When fuzzing input parameter revealed when user-generated payloads like ‘<script>alert('XSS')</script>’ or ‘<h1>IsThisSecure</h1>’ were transmitted into the database and then rendered as actual HTML elements. This reveal major flaw, the implementation of output encoding filters was disabled in `dashboard.html`. If this code push to production, malicious JavaScript strings could be saved into the central application database, making authenticated users vulnerable to session/cookie hijacking.
- **Improper Exception Handling:** Using Burp Suite Repeater tab, we sent mismatched validation parameters directly to the account register handler. When the application

code process unmatched password strings, it triggered unhandled runtime error exception resulting in an unstable HTTP/1.1 500 Internal Server Error server response. In a real enterprise environment, unhandled execution crashes can give attackers valuable information about database operation and backend validation parameters.

10. Conclusion

This project successfully completed the design, development, security verification and release phases of a production-hardened, multi-tier Task Management Web Application build on the Python Django framework. By implementing security first design pattern from day one, our team successfully create custom input verification forms, role-based database queries and multi-tenant isolation layers. Automated static analysis with Snyk help us track down environment configuration flaws early, while continuously manual assessment and dynamic testing using network proxies to find and fix application layout and data parsing problems before finalizing the system.

Future Enhancements

To keep improving security and keep up with compliance standards, there are several future enhancements are recommended for future versions:

- **Automated DAST in the GitHub Actions Pipeline:** Transition from manually executed tool scans to automated runtime scanning jobs that run on every commit.
- **Robust Multi-Factor Authentication (MFA):** Integrate standard MFA such as Time-Based One-Time Passwords (TOTP), to strengthen authentication against automated credential threats.
- **Comprehensive Content Security Policies (CSP):** Implement strict HTTP CSP response headers to control where scripts can load from and provide additional layer of defence against accidental injection risks.

11. References

- Open Web Application Security Project (OWASP). (2021). OWASP Top 10:2021 - The Critical Security Risks to Web Applications. Retrieved from https://owasp.org/www-project-top-ten/
- Open Web Application Security Project (OWASP). (2023). OWASP Application Security Verification Standard (ASVS) 4.0.3. Retrieved from https://owasp.org/www-project-application-security-verification-standard/

- Django Software Foundation. (2026). Django Documentation: Security in Django. Retrieved from https://docs.djangoproject.com/en/stable/topics/security/

12. Appendices

Appendix A: Code Snippets

```python
class RegisterForm(forms.Form):
    username = forms.CharField(
        max_length=150,
        validators=[
            RegexValidator(
                regex='^[a-zA-Z0-9_]+$',
                message='Username can only contain letters, numbers, and underscores'
            )
        ]
    )
    email = forms.EmailField()
    password = forms.CharField(
        widget=forms.PasswordInput,
        min_length=8,
        validators=[MinLengthValidator(8, 'Password must be at least 8 characters')]
    )
```

```python
SESSION_COOKIE_AGE = 1800
SESSION_COOKIE_HTTPONLY = True
SESSION_EXPIRE_AT_BROWSER_CLOSE = True
CSRF_COOKIE_HTTPONLY = True
```

```python
@login_required
def task_edit(request, pk):
    task = get_object_or_404(Task, pk=pk)
    if request.user.role != 'admin' and task.created_by != request.user:
        messages.error(request, 'You do not have permission to edit this task')
        return redirect('dashboard')
```

```python
def login_view(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')

        if not username or not password:
            messages.error(request, 'Please enter both username and password')
            return render(request, 'tasks/login.html')

        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)
            log_audit(user, 'LOGIN_SUCCESS', request)
            messages.success(request, f'Welcome back, {user.username}!')
            return redirect('dashboard')
        else:
            log_audit(None, 'LOGIN_FAILED', request, f'Failed login attempt for {username}')
            messages.error(request, 'Invalid username or password')
```

```python
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
        'OPTIONS': {
            'min_length': 8,
        }
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]
```

```python
@login_required
def profile_edit(request):
    if request.method == 'POST':
        form = ProfileForm(request.POST, request.FILES, instance=request.user)
        if form.is_valid():
            form.save()
            log_audit(request.user, 'PROFILE_UPDATED', request)
            messages.success(request, 'Profile updated successfully!')
            return redirect('profile')
    else:
        form = ProfileForm(instance=request.user)
    return render(request, 'tasks/profile_edit.html', {'form': form})
```

```python
class ProfileForm(forms.ModelForm):
    class Meta:
        model = CustomUser
        fields = ['first_name', 'last_name', 'email', 'phone', 'profile_picture']
```

```python
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

```python
class AuditLog(models.Model):
    """Audit log for security events"""
    ACTION_CHOICES = (
        ('LOGIN_SUCCESS', 'Login Success'),
        ('LOGIN_FAILED', 'Login Failed'),
        ('LOGOUT', 'Logout'),
        ('TASK_CREATED', 'Task Created'),
        ('TASK_UPDATED', 'Task Updated'),
        ('TASK_DELETED', 'Task Deleted'),
        ('ROLE_CHANGED', 'Role Changed'),
        ('PROFILE_UPDATED', 'Profile Updated'),
    )
```

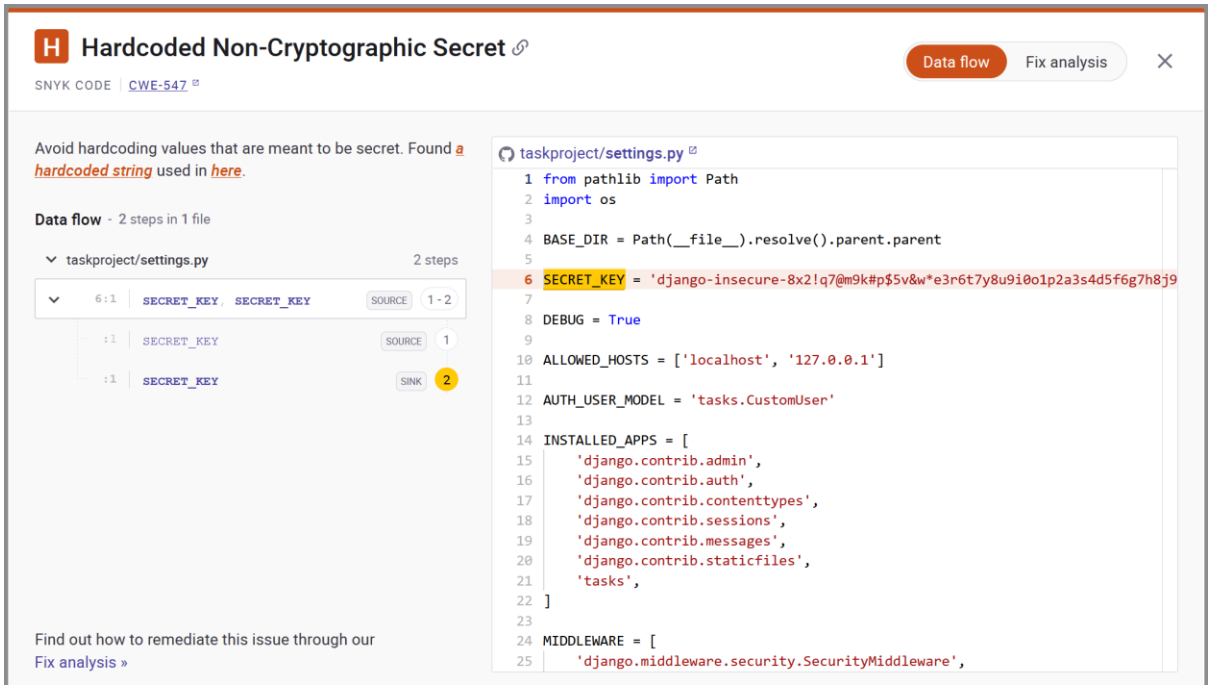
```python
def log_audit(user, action, request, details=''):
    AuditLog.objects.create(
        user=user,
        action=action,
        ip_address=request.META.get('REMOTE_ADDR'),
        details=details
    )
```

```html
<tr>
    <td style="font-weight: 600;">{{ task.title }}</td>
    <td style="color: #6b7280;">{{ task.description|truncatechars:50 }}</td>
    <td>
```

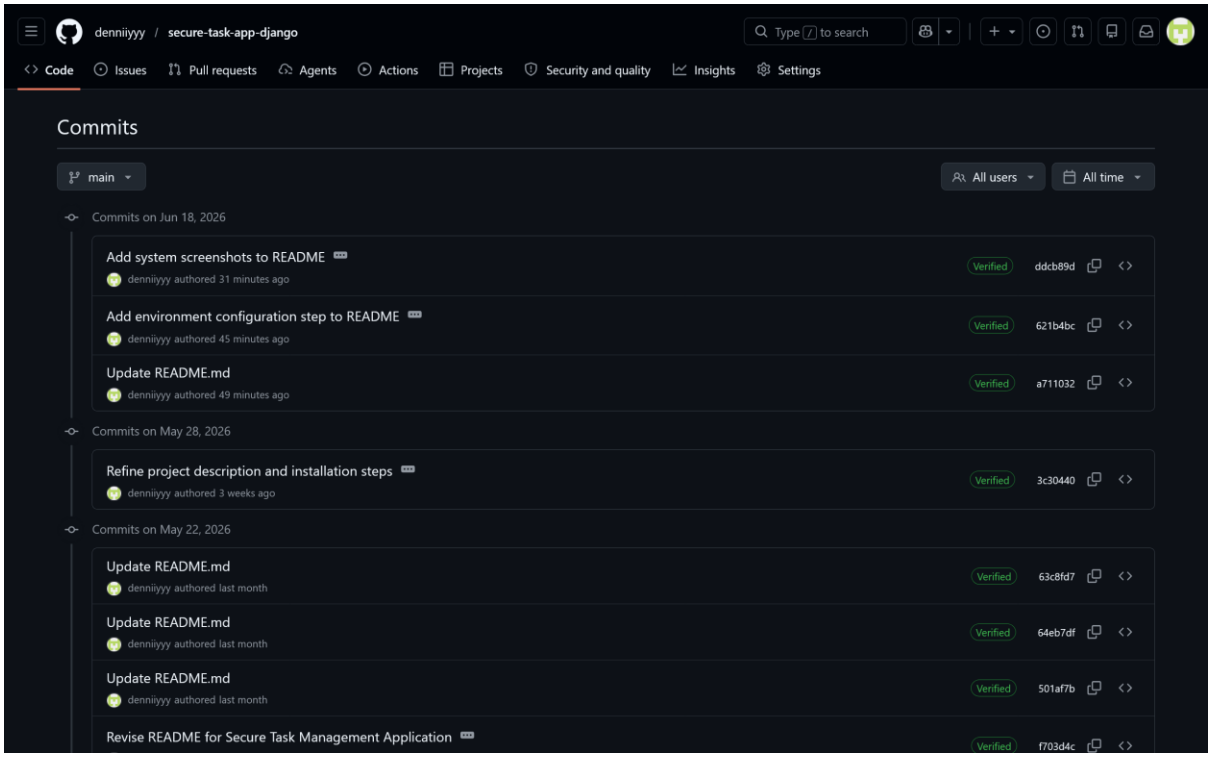
Appendix B: Scan Result

```
C:\Users\Denni\Documents\UNIKL\SEM2\SSD\Group\secure-task-app>venv\Scripts\activate

(venv) C:\Users\Denni\Documents\UNIKL\SEM2\SSD\Group\secure-task-app>snyk test --file
=requirements.txt

Testing C:\Users\Denni\Documents\UNIKL\SEM2\SSD\Group\secure-task-app...

Organization:      denniiyyy
Package manager:   pip
Target file:       requirements.txt
Project name:      secure-task-app
Open source:       no
Project path:      C:\Users\Denni\Documents\UNIKL\SEM2\SSD\Group\secure-task-app
Licenses:          enabled

✔Tested 5 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

Appendix C: Static Analysis Report



Appendix D: GitHub Commit History Screenshots



Appendix E: README.md screenshot

README

Secure Task Management Application (secure-task-app)

An implementation of an injection-free, role-based Task Management web application built using secure-by-design architectural paradigms. This project serves as the major assessment for **IKB 21503: Secure Software Development** at UniKL MIIT.

1. Project Description

The **Secure Task Management Application** is a collaborative system for managing workflows, task assignments and tracking progress securely. This application is build using the Python **Django** framework and focuses on addressing OWASP Top 10 vulnerabilities.

Rather than relying purely on passive framework automation, this system actively integrates robust input sanitization pipelines, strict multi-tenant data boundaries, structural session security and continuous third-party vulnerability remediation to guarantee the confidentiality, integrity, and availability of system state profiles.

2. Security Features Summary

This application implements the following security controls aligned with OWASP ASVS:

- **Input Validation:** Strict regex and built-in validators to prevent injection attacks.
- **Authentication & Session Security:** Enforced session timeouts (30 mins), HttpOnly cookies and strict password complexity validators.
- **Access Control (RBAC):** Strict data isolation between standard "Users" and "Admins" to prevent Insecure Direct Object Reference (IDOR).
- **Sensitive Data Protection:** Cryptographic secrets and debug variables are strictly decoupled from version control using `.env` files.
- **Output Encoding:** Enforced Django template auto-escaping to mitigate Stored XSS and HTML Injection vectors.
- **Audit Logging:** Centralized recording of security events (logins, task modifications, profile updates) with IP tracking.

3. Dependencies

This project operates on Python 3.10+ and Django. All third-party dependencies are tracked in `requirements.txt` and have been cleared by **Snyk** Software Component Analysis (SCA) to ensure supply chain security. Core packages include:

- `Django`
- `django-environ` (For secure secret management)

4. Installation Steps

Follow these sequential procedures to deploy the development environment locally:

1. **Clone the Project Repository:**

```
git clone https://github.com/denniiyyy/secure-task-app-django.git
cd secure-task-app-django
```

2. Initialize Local Virtual Environment:

```
python -m venv venv
```

3. Activate Environment Context:

```
venv\Scripts\activate
```

4. Provision Third-Party Modules:

```
pip install -r requirements.txt
```

5. Environment Configuration Rename the provided `.env.example` file to `.env`

6. Execute Relational Schema Migrations:

```
python manage.py migrate
```

7. How to Run the App:

```
python manage.py runserver
```